



《AI大模型Ragent项目》——抢不到许可请求该等还是该拒？

来自：拿个offer-开源&项目实战



马丁

2026年05月14日 22:04

开篇引言

上一篇讲了排队限流的同步路径——streamChat 直接委托 ChatQueueLimiter.enqueue() 入队写 ZSET、tryAcquireIfReady 四阶段抢占 (isPending 状态检查 / availablePermits 判断 / Lua 脚本 claim / RPermitExpirableSemaphore.tryAcquire)。镜头停在了在线教育公司促销那一刻：10 个请求同时打到两台机器，前 3 个走完四阶段拿到许可、提交到 chatEntryExecutor 跑业务。看起来挺顺。但故事只讲了一半。第 4 到第 10 个请求呢？他们调 tryAcquireIfReady 时阶段 2 就被挡住了——availablePermits = 0，许可全在前 3 个手里。返回 false 之后，进入了上一篇没展开的 scheduleQueuePoll。这 7 个请求安安静静地躺在 ZSET 里，前端的「思考中...」转圈圈还在转。问题就来了：

他们怎么知道许可释放了？——前 3 个请求中，请求 1 是节点 B 上的，跑了 25 秒终于跑完释放了一个许可。节点 A 上等着的请求 4 怎么得到通知？

不通知就只能轮询吗？多久轮一次？——轮询太勤 Redis 受不了，轮询太疏用户等不起，得有个折中方案

等了 20 秒还没轮到自己怎么办？——总不能把 SSE 连接挂着不响应，得给用户一个明确的交代。但只关连接也太粗暴了，前端只能看到一个空白

如果许可被进程崩溃带走永远还不回来，集群总并发就这么一直降下去吗？——必须有兜底机制

应用关闭那一瞬间，还在排队的请求该怎么收尾？——daemon 线程虽然 JVM 退出会自动停，但留下半中断的状态总是不优雅

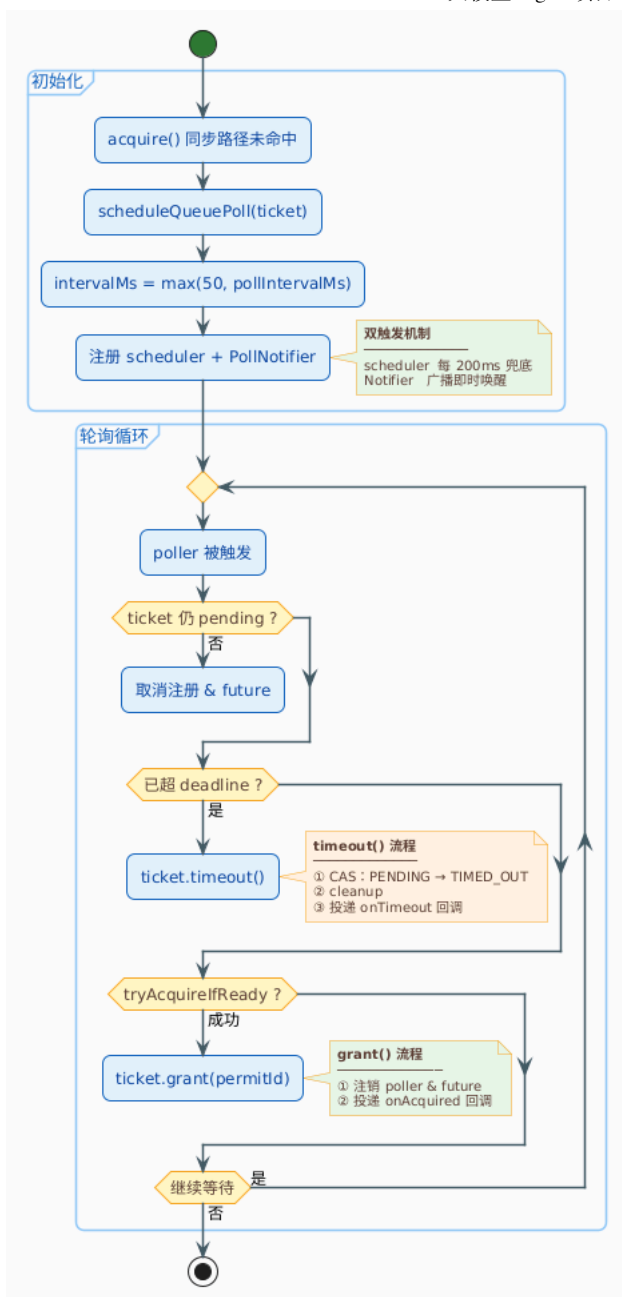
这一篇就来回答这五个问题。技术上对应五段内容——scheduleQueuePoll 调度轮询、Pub/Sub 跨集群广播 + PollNotifier 防惊群、超时拒绝写会话记忆、Ticket.cleanup() 幂等清理 + lease 兜底、Spring destroyMethod 优雅关闭。讲完之后再回头看排队限流的全链路图，最后给整个 AI 知识问答 18 篇做一次完整收束。

需要提醒一下：上一篇结尾已经讲过，异步等待路径 (scheduleQueuePoll)、PollNotifier、Ticket.timeout() 等逻辑现在都住在 FairDistributedRateLimiter 里，ChatQueueLimiter 只负责 SSE 业务编排。超时拒绝的业务处理 (写会话记忆、推 SSE 事件) 通过 AcquireRequest.onTimeout 回调交给 ChatQueueLimiter。本篇会反复在这两层之间切换，先搞清楚谁干什么，后面读起来会顺很多。听起来很合理，但实际跑起来你会发现细节多得吓人——光「通知所有等待者」一句话，背后就藏着跨集群广播、防惊群、防丢失、自动清理、优雅关闭好几层问题。这是排队限流「看起来很简单实际很复杂」的地方，本篇要把这几层一层一层揭开。

等待路径的入口：scheduleQueuePoll

1. 一张图看完整等待流程

先用一张活动图把单个 poller 的生命周期串起来。



整个生命周期就三个出口：被取消、超时、抢到。Ticket 状态机保证三个出口互斥——PENDING 只能单次 CAS 转到 CANCELLED、TIMED_OUT、GRANTED 之一。无论走哪个出口，Ticket.cleanup() 都会被执行：幂等地移队、删除 entry 标记、释放许可（仅非 GRANTED 状态下）、注销 poller、取消 future。

2. scheduler 的角色

scheduleQueuePoll 用的调度器是一个小线程池，在 FairDistributedRateLimiter 的构造函数里创建：

```
String threadPrefix = name.replace(':', '_');
int schedulerSize = Math.min(4, Math.max(2, Runtime.getRuntime().availableProcessors() / 2));
AtomicInteger threadCounter = new AtomicInteger();
this.scheduler = new ScheduledThreadPoolExecutor(schedulerSize, r -> {
    Thread t = new Thread(r);
    t.setName(threadPrefix + "_scheduler_" + threadCounter.incrementAndGet());
    t.setDaemon(true);
    return t;
});
```

线程数根据 CPU 核数动态计算—— $\text{Math.min}(4, \text{Math.max}(2, \text{cpu} / 2))$ ，最少 2 个、最多 4 个。线程名用限流器的 name 做前缀（冒号替换成下划线）加递增编号，比如 rag_global_chat_scheduler_1、rag_global_chat_scheduler_2——一眼就能在线程 dump 里看到是哪个限流器的第几个调度线程。

为什么不是单线程？因为这个调度器不只跑 poller 的周期任务，还被 PollNotifier 复用来跑广播触发的扫描。促销时刻广播和周期轮询同时在跑，2~4 个线程能更及时地消化任务，避免单线程排队延迟。但线程数也不需要太多——poller 本身做的事很轻：查状态、查时间、调

tryAcquireIfReady, 几毫秒到几十毫秒就完事。

设为 daemon 的理由是不阻塞 JVM 退出——主线程没了 daemon 线程会自动收掉。但即使如此, 本篇后面会讲 stop() 还是会主动 shutdown 一下, 让正在跑的 poller 走完, 不留半中断状态。

3. scheduleQueuePoll() 代码逐段解读

重构后 scheduleQueuePoll 变得非常简洁——所有状态都收拢在 Ticket 里, 不再需要把 queue、requestId、permitRef、cancelled、question、conversationId、userId、emitter、onAcquire 九个参数逐个传进来。完整代码:

```
private void scheduleQueuePoll(Ticket ticket) {
    int interval = Math.max(50, pollIntervalMsSupplier.getAsInt());
    Runnable poller = () -> {
        if (!ticket.isPending()) {
            ticket.unregisterFromNotifier();
            ticket.cancelFutureQuietly();
            return;
        }
        if (System.currentTimeMillis() > ticket.deadline) {
            ticket.timeout();
            return;
        }
        tryAcquireIfReady(ticket);
    };
    ticket.future = scheduler.scheduleAtFixedRate(poller, interval, interval, TimeUnit.MILLISECONDS);
    pollNotifier.register(ticket.requestId, poller);
}
```

下面分四块讲。

3.1 intervalMs 和 deadline

```
int interval = Math.max(50, pollIntervalMsSupplier.getAsInt());
```

interval 是轮询间隔。从配置取 poll-interval-ms (默认 200), 但用 Math.max(50, ...) 兜底——配低于 50ms 的话会强制拉到 50ms。为什么? 50ms 是经验值, 比这个更短的轮询会让 Redis 命令频繁被触发, 完全没必要——许可释放是事件级别的事, 跑得再勤也快不过 Pub/Sub 的即时唤醒。

deadline 不在这里算——它已经在 Ticket 构造时就算好了:

```
Ticket(AcquireRequest req) {
    this.req = req;
    this.deadline = System.currentTimeMillis() + req.maxWaitMillis();
}
```

deadline 是绝对时间戳——「我最晚等到几点几分几秒几毫秒」。这样比每次比较「已等待时长 < max-wait-seconds」更直观, poller 内部一次 System.currentTimeMillis() > ticket.deadline 比较就完事。

回到在线教育公司: max-wait-seconds=20, poll-interval-ms=200。也就是说每个等待者最多等 20 秒, 期间每 200ms 兜底跑一次 poller, 加上 Pub/Sub 即时唤醒。20 秒 / 200ms = 100 次轮询, 对单个等待者来说, 开销可控。

3.2 定义 poller lambda 的三个分支

poller 是一个 Runnable, 每次被触发都跑一遍这段逻辑。三个 if 分支对应三个出口:

第一个分支处理取消或已完成:

```
if (!ticket.isPending()) {
    ticket.unregisterFromNotifier();
    ticket.cancelFutureQuietly();
    return;
}
```

`isPending()` 内部就是 `state.get() == State.PENDING`——只有还在排队的 `Ticket` 才有继续抢占的必要。如果状态已经不是 `PENDING` (可能是 `CANCELLED`——前端关连接触发 `ticket.cancel()`；也可能是 `GRANTED`——被别的 `poller` 触发路径抢先成功了)，直接注销 + 取消调度走人。

和老版本用 `cancelled.get()` 不同，`Ticket` 状态机覆盖了更多终态——不只是外部取消，`grant` 成功后的清理也会让 `isPending()` 返回 `false`，避免重复执行。

第二个分支处理 `deadline`：

```
if (System.currentTimeMillis() > ticket.deadline) {
    ticket.timeout();
    return;
}
```

这是超时拒绝的入口。`ticket.timeout()` 内部做三件事：

```
void timeout() {
    if (!state.compareAndSet(State.PENDING, State.TIMED_OUT)) {
        return;
    }
    cleanup();
    submitSafely(req.onTimeout(), "onTimeout");
}
```

1. `CAS PENDING → TIMED_OUT`：抢占终态。如果 `CAS` 失败说明已经被 `cancel()` 或 `grant()` 抢先了，直接 `return`
2. `cleanup()`：幂等清理——从 `ZSET` 移除自己、删除 `entry` 标记、释放已持有的许可（若有，非 `GRANTED` 状态下）、广播通知、注销 `poller`、取消 `future`
3. `submitSafely(req.onTimeout(), "onTimeout")`：把 `onTimeout` 回调提交到 `onAcquiredExecutor` 执行。在 `Chat` 场景里这个回调就是 `() -> handleReject(question, conversationId, emitter)`——写会话记忆 + 推 `SSE` 拒绝事件

注意 `cleanup()` 已经包含了注销 `poller` 和取消 `future` 的动作，所以 `poller lambda` 里不需要再显式调一遍——`timeout()` 一个方法搞定一切。

第三个分支处理抢到：

```
tryAcquireIfReady(ticket);
```

调上一篇讲过的 `tryAcquireIfReady` 四阶段。返回 `true` 说明许可到手了——`ticket.grant()` 内部已经注销 `poller`、取消 `future`、提交业务到 `chatEntryExecutor`，`poller` 的使命结束。返回 `false` 说明还没轮到自己（许可没空 / `claim` 失败 / 二次 `isPending` 短路），`poller` 不动作，等下次触发再试。

3.3 `ticket.future` 赋值

```
ticket.future = scheduler.scheduleAtFixedRate(poller, interval, interval, TimeUnit.MILLISECONDS);
```

和老版本的 `ScheduledFuture<?>[] futureRef = new ScheduledFuture<?>[1]` 一元素数组 `trick` 相比，现在直接把 `future` 存在 `Ticket` 的 `volatile` 字段上，更直观。`Ticket` 是 `FairDistributedRateLimiter` 的内部类，`poller lambda` 闭包捕获的是 `ticket` 引用（effectively-final），通过 `ticket.future` 间接访问 `ScheduledFuture`，绕过了 `lambda` 闭包的 `effectively-final` 限制。

`cancelFutureQuietly()` 是 `Ticket` 的实例方法，做幂等取消：

```
void cancelFutureQuietly() {
    ScheduledFuture<?> f = future;
    if (f != null && !f.isCancelled()) {
        f.cancel(false);
    }
}
```

`scheduleAtFixedRate(task, initialDelay, period, unit)`——首次延迟 `interval` 后跑，之后每隔 `interval` 跑一次。注意这里 `initialDelay` 也是 `interval` 而不是 `0`——因为同步路径的 `tryAcquireIfReady` 刚跑过一次失败了，再立刻跑一次浪费，等 `200ms` 之后再试。

3.4 pollNotifier.register

最后一行：

```
pollNotifier.register(ticket.requestId, poller);
```

把 poller 注册到广播通知器。这就是「两条触发路径合并」的关键——同一个 poller，既被 scheduler 周期性调用，也被 PollNotifier 在收到广播时调用。两个路径都不会丢，重复调用也是安全的（poller 内部三个分支都是状态判断）。

4. 三个出口的清理动作

把三个出口的清理动作整理成一张表：

出口	触发条件	共用清理	额外动作
取消（CANCELLED）	客户端断连 / SSE 超时 / 推送出错 → ticket.cancel()	ticket.cleanup()（移队 + 删 entry 标记 + 释放 permit + 广播 + 注销 poller + 取消 future）	无（已被取消，不触发业务回调）
超时（TIMED_OUT）	等待超过 max-wait-seconds → ticket.timeout()	ticket.cleanup()	submitSafely(onTimeout)：写记忆 + 推 SSE reject
抢到（GRANTED）	tryAcquireIfReady → ticket.grant()	ticket.unregisterFromNotifier() + cancelFutureQuietly()	提交 onAcquired 到 chatEntryExecutor，permit 由 try/finally 包装管理

和老版本相比，最大的变化是 cleanup 被统一收进了 Ticket.cleanup() 一个方法——移队、删除 entry 标记、释放许可（非 GRANTED 状态下）、广播、注销、取消 future，所有子操作各自幂等，调多少次都安全。三个出口不再需要手动拼凑清理逻辑，减少了遗漏风险。

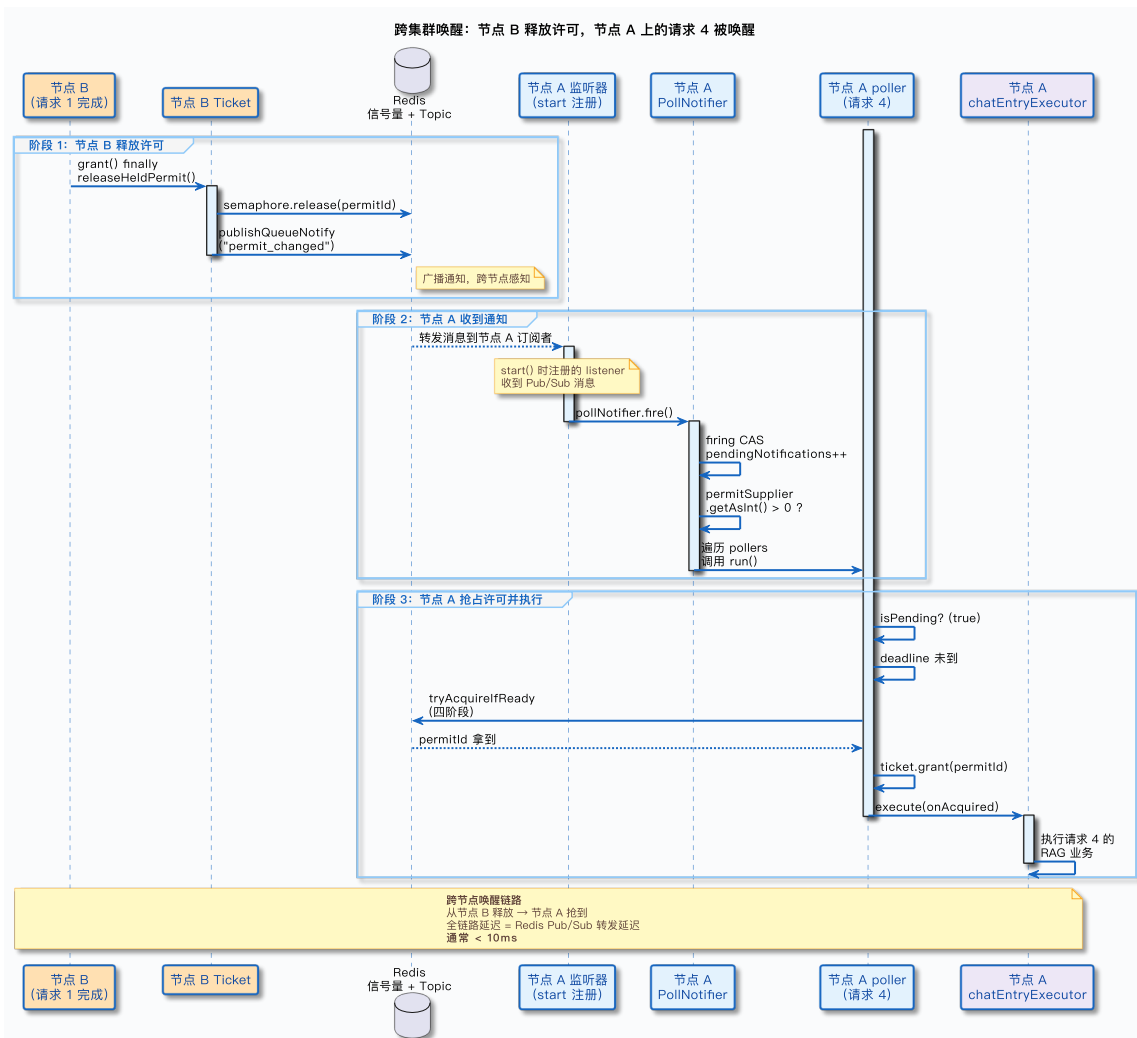
这种「状态机 + 统一 cleanup」的模式是写多线程异步代码的好习惯。CAS 保证终态互斥（回调最多触发一次），cleanup 幂等保证资源不泄漏（调多少次都安全），两者配合让并发代码变得可推理。

Pub/Sub 跨集群广播：节点 A 怎么知道节点 B 释放了许可

scheduleQueuePoll 注册了两条触发路径，scheduler 那条已经讲完。剩下一条是 PollNotifier——靠 Redis Pub/Sub 跨集群唤醒。这一节先把广播链路讲清楚，下一节再深入 PollNotifier 的防惊群设计。

1. 一张图看跨集群唤醒

回到在线教育公司的场景：节点 A 上的请求 4 在排队，节点 B 上的请求 1 跑完业务释放了一个许可。这时候节点 A 上的请求 4 怎么得到通知？



整条链路看来，关键节点是：节点 B 请求完成 → grant() 包装 Runnable 的 finally 块调 releaseHeldPermit() 释放许可并 publish 一条消息 → Redis 把这条消息转发给所有订阅者 → 节点 A 的 listener 收到 → 调 pollNotifier.fire() → 触发节点 A 上所有等待中的 poller 跑一遍 → 请求 4 的 poller 跑完 tryAcquireIfReady 拿到许可。

细心的读者可能会问：请求 1 正常完成后许可是怎么释放的？在 Ticket 模型里，grant() 把 onAcquired 包在了 try/finally 里——业务执行完毕后 finally 块调 releaseHeldPermit() 释放许可并广播。同时 cancelBinder 把 ticket::cancel 绑定到了 emitter.onCompletion / onTimeout / onError 三个回调，cancel() 的 cleanup() 会做其他清理（移队、删 entry 标记、注销 poller 等），但在 GRANTED 状态下不会释放许可——许可的生命周期已经由 grant() 的包装接管。

2. start(): 初始化信号量 + 注册监听器

老版本用 @PostConstruct subscribeQueueNotify() 在 ChatQueueLimiter 上做初始化。重构后这些逻辑全部移到了 FairDistributedRateLimiter.start() 里，通过 Spring 的 initMethod 触发：

```
// ChatRateLimiterConfig.java
@Bean(initMethod = "start", destroyMethod = "stop")
public FairDistributedRateLimiter chatRateLimiter(RedissonClient redissonClient,
                                                  RAGRateLimitProperties rateLimitProperties) {
    return new FairDistributedRateLimiter(
        CHAT_LIMITER_NAME,
        redissonClient,
        rateLimitProperties::getGlobalMaxConcurrent,
        rateLimitProperties::getGlobalLeaseSeconds,
        rateLimitProperties::getGlobalPollIntervalMs
    );
}
```

start() 方法：

```
public void start() {
    if (!started.compareAndSet(false, true)) {
```

```
        return;
    }
    redissonClient.getPermitExpirableSemaphore(semaphoreKey)
        .trySetPermits(maxPermitsSupplier.getAsInt());
    RTopic topic = redissonClient.getTopic(notifyTopicKey);
    notifyListenerId = topic.addListener(String.class, (channel, msg) -> pollNotifier.fire());
}
```

两件事（加一个防重入守卫）：

`started.compareAndSet(false, true)`：CAS 守卫，防止重复初始化。`started` 是 `AtomicBoolean`，保证整个生命周期只初始化一次

`trySetPermits(maxPermitsSupplier.getAsInt())`：初始化信号量许可数。`trySetPermits` 自身幂等——Redis 里已有许可数就跳过。放在 `start()` 避免了每次 `tryAcquirePermit` 都多一次 Redis 往返

`topic.addListener(String.class, ...)`：注册监听器，绑定到 Redis Topic。监听回调里只做一件事——调 `pollNotifier.fire()`

为什么用 `initMethod` 而不是 `@PostConstruct`？因为 `FairDistributedRateLimiter` 是一个通用组件，没有 `@Component` 注解。通过 `initMethod` / `destroyMethod` 把生命周期管理交给配置类，组件本身保持 POJO——换个框架也能用。

`notifyListenerId` 保存监听器 ID，后面 `stop()` 时用来 `removeListener`，避免应用关闭后还在处理消息。

3. publishQueueNotify()

发布很简单，一行：

```
private void publishQueueNotify() {
    redissonClient.getTopic(notifyTopicKey).publish("permit_changed");
}
```

消息内容是字符串 `permit_changed`，本身不携带任何信息——监听端只关心「有事发生」，不关心是哪个许可被释放、被谁释放。`fire()` 会触发本地所有等待中的 `poller` 都跑一遍，让它们自己去看自己的状态、自己去抢许可。

什么时候调 `publishQueueNotify`？在新的 `Ticket` 模型下，调用点更加集中：

时机	触发动作	目的
<code>Ticket.cleanup()</code> 移队或释放 permit 后	<code>queue.remove</code> 成功或 permit 释放成功（仅非 GRANTED 状态）	让其他等待者知道队列或许可状态变了
<code>Ticket.releaseHeldPermit()</code>	<code>grant()</code> 包装的 <code>finally</code> 块或提交失败时显式调用	业务执行完毕后让等待者抢释放的许可
<code>tryAcquireIfReady</code> claim 成功 acquire 失败重入队后	<code>queue.add(claimedScore, requestId)</code> 完成	让队列推进，被跳过位置的等待者可以来抢
<code>tryAcquireIfReady</code> acquire 成功准备 grant 前	<code>tryAcquirePermit</code> 返回 <code>permitId</code>	让其他等待者再尝试一轮（可能还有许可剩）
<code>Ticket.grant()</code> CAS 失败释放 permit 后	<code>releasePermitQuietly</code> 完成	让等待者来抢释放的许可

和老版本分散在 `releaseOnce`、`releasePermit`、`tryAcquireIfReady`、超时分支等多个地方不同，新版本把广播收进了 `Ticket.cleanup()` 和 `Ticket.releaseHeldPermit()` 两个方法——`cleanup()` 内部判断「移队成功 或 释放了 permit」时才广播，避免无意义的空广播。

注意 `publish` 是节点本地行为——节点 B 调 `publishQueueNotify()` 之后，Redis 会把消息转发给所有订阅这个 topic 的节点，包括节点 B 自己。所以节点 B 上的等待者也会被自己触发的广播唤醒。这种「自广播」看起来有点冗余，但实际是好事——节点 B 上可能也有其他等待者，和「节点 B 释放许可、同节点的其他等待者要抢」语义完全一致。

4. 为什么 Pub/Sub 之外还需要轮询

这里要回答一个常见疑问：既然有 Pub/Sub 即时唤醒，为什么还需要 `scheduleAtFixedRate` 周期轮询兜底？

Pub/Sub 不是 100% 可靠的。Redis Pub/Sub 是「fire and forget」——发出去就不管了，不持久化、不重试。下面几种场景都会导致广播丢失：

- Redis 主从切换期间：主节点挂了，哨兵切换到从节点。这中间 `publish` 会失败，订阅连接会断
- 订阅连接断开重连之前：网络抖动让订阅 socket 断了，Redisson 会自动重连，但重连之前的广播已经丢了
- 节点新启动时已经有等待中的请求：你刚启动一个新节点，正好这时候老节点上有等待者，并且老节点上有许可释放——新节点的 listener 还没注册完成，这条广播也接不到

GC 卡顿期间：节点 A 在做 Full GC 卡了 500ms，期间的广播虽然 listener 在注册，但回调线程也卡着没法处理。等 GC 结束消息已经过期被 Redis 丢弃

任何一种场景下，如果只靠 Pub/Sub，等待者会卡死——明明有许可可以抢但没人通知它。轮询是保险——即使广播全丢，每 200ms 也会主动跑一次 tryAcquireIfReady，最多多等 200ms 就能抢到。

反过来也成立——光有轮询不够。轮询响应慢（最坏 200ms），并且把 Redis 调用密度搞高（10 个等待者每 200ms 跑一次，就是 50 次/秒）。Pub/Sub 是即时的，正常情况下许可释放后 Redis 网络转发延迟几毫秒就触达，等待者立刻能抢到。

两者结合是工程上的最佳实践：**Pub/Sub 走快路径，轮询走慢路径兜底**。99% 的时候 Pub/Sub 跑得飞快，轮询基本是空跑；万一 Pub/Sub 失效，轮询挺身而出。

类似的「主动通知 + 被动轮询」组合在分布式系统里非常常见。注册中心（Eureka / Nacos）服务发现就是 watcher 通知 + 定期拉取兜底；MQ 消费者收到 push 也兼有 pull 兜底；K8s 的 Informer 也是 watch + list 组合。原理都一样——通知机制不可靠时，要有一条独立路径保证最终一致性。

PollNotifier：防惊群与去抖的核心

讲完了广播链路，下面深入 PollNotifier。这是排队限流里最难的一段代码——fire() 方法只有 20 多行，但藏着三层防御。

1. 整体结构

PollNotifier 住在 FairDistributedRateLimiter 内部，作为一个 private static final class。先看字段定义：

```
private static final class PollNotifier {

    private final IntSupplier permitSupplier;
    private final Executor executor;
    private final ConcurrentHashMap<String, Runnable> pollers = new ConcurrentHashMap<>();
    private final AtomicBoolean firing = new AtomicBoolean(false);
    private final AtomicInteger pendingNotifications = new AtomicInteger(0);

    PollNotifier(IntSupplier permitSupplier, Executor executor) {
        this.permitSupplier = permitSupplier;
        this.executor = executor;
    }
    // ...
}
```

四个核心组件：

组件	类型	作用
pollers	ConcurrentHashMap<String, Runnable>	当前节点所有等待中的 poller，按 requestId 索引
firing	AtomicBoolean	防止多个广播任务同时跑，CAS 串行化
pendingNotifications	AtomicInteger	计数挂起的通知，避免任务级雪崩
executor	外部传入的 Executor	跑广播触发的 poller 遍历，复用 FairDistributedRateLimiter 的 scheduler

和老版本相比，最大的变化是 PollNotifier 不再自己创建线程池——构造时传入外部的 executor（就是 FairDistributedRateLimiter 的 scheduler），让广播触发的扫描和周期性轮询共享同一个调度器。这样整个限流器只有一个线程池加一个外部业务池，资源更紧凑。

2. register / unregister

注册和注销很直白：

```
void register(String requestId, Runnable poller) {
    pollers.put(requestId, poller);
}

void unregister(String requestId) {
```

```
pollers.remove(requestId);
}
```

register 用 requestId 当 key，把 poller 直接塞进 Map。后续 fire() 遍历这个 Map 触发所有 poller。
unregister 调 ConcurrentHashMap.remove——不存在的 key 删除是空操作，多次调用幂等。这就是为什么 Ticket 的各种清理方法都敢直接调 unregister，不需要先判存在。

3. fire()：最核心的方法

完整代码：

```
void fire() {
    pendingNotifications.incrementAndGet();
    if (!firing.compareAndSet(false, true)) {
        return;
    }
    executor.execute(() -> {
        do {
            pendingNotifications.set(0);
            try {
                if (permitSupplier.getAsInt() <= 0) {
                    break;
                }
                for (Runnable poller : pollers.values()) {
                    try {
                        poller.run();
                    } catch (Exception ex) {
                        log.debug("poller 执行异常", ex);
                    }
                }
            } finally {
                firing.set(false);
            }
        } while (pendingNotifications.get() > 0 && firing.compareAndSet(false, true));
    });
}
```

20 多行，藏了三层防御。

3.1 第一道防线：CAS 串行化

```
pendingNotifications.incrementAndGet();
if (!firing.compareAndSet(false, true)) {
    return;
}
```

进来先 pendingNotifications.incrementAndGet() 加——告诉「正在跑的那个任务（如果有）」还有事要处理。然后 CAS 抢 firing 标志位：

CAS 成功 (false → true)：当前线程拿到了「广播任务执行权」，继续往下提交任务

CAS 失败：说明已经有任务在跑了，自己什么都不做直接 return。挂起的通知数已经累加了，那个跑着的任务会在跑完后看到 pendingNotifications > 0 再跑一轮

这个 CAS 是关键——它保证同一时刻最多只有一个广播扫描任务在跑。回到在线教育公司：促销时刻一秒内可能连续来好几次广播（节点 A 释放、节点 B 释放、节点 A 又有请求 claim 后广播），如果不串行化，一秒内就会启动 4 个任务，每个任务遍历 N 个 poller，总开销翻 4 倍——但实际上做的事是一样的（让所有 poller 跑一遍）。

3.2 第二道防线：pendingNotifications

CAS 成功的线程提交任务到 executor：

```

executor.execute() -> {
    do {
        pendingNotifications.set(0);
        try {
            if (permitSupplier.getAsInt() <= 0) {
                break;
            }
            for (Runnable poller : pollers.values()) {
                try {
                    poller.run();
                } catch (Exception ex) {
                    log.debug("poller 执行异常", ex);
                }
            }
        } finally {
            firing.set(false);
        }
    } while (pendingNotifications.get() > 0 && firing.compareAndSet(false, true));
});

```

do-while 结构。进入循环就 `pendingNotifications.set(0)`——「我要开始干活了，把之前累积的通知清零，从现在开始累积的算下一轮」。然后跑一遍 poller，跑完 `firing.set(false)` 释放执行权。

退出条件是 `pendingNotifications.get() > 0 && firing.compareAndSet(false, true)`——「跑期间如果还有新通知来，重新抢回执行权再跑一轮」。

这是「火上加柴」的语义。打个比方：你在烧水，水开了你就把火关了。但烧水的过程中如果有人又往里加了一些冷水（新通知），你看到水还没真的开（`pendingNotifications > 0`），就把火重新打开继续烧。这种结构避免了「烧到一半再启动一个新的炉子」的浪费。

具体收益：促销时刻一秒内连续来 5 次广播。第一个广播 CAS 成功启动任务，跑期间又来了 4 个广播，每个都让 `pendingNotifications` 加一但 `firing.compareAndSet` 失败直接返回。第一个任务跑完，看到 `pendingNotifications=4 > 0`，CAS 抢回执行权，再跑一轮。这一轮把 4 个挂起的通知一次性消化掉。最终 5 个广播只触发了 2 轮 poller 遍历——遍历次数远小于广播次数。

和老版本还有一个区别：新版本在 poller 遍历时加了 try-catch 包裹每个 `entry.poller().run()` 调用。这样某个 poller 抛异常不会中断整轮遍历——其他 poller 该跑还是跑。异常被 `log.debug` 静默记录，不影响主流程。

3.3 第三道防线：空轮次跳过

循环体里这一行是第三道防线：

```

if (permitSupplier.getAsInt() <= 0) {
    break;
}

```

`permitSupplier` 是构造时传进来的 `this::availablePermits` 方法引用，每次调一次 Redis 查可用许可数。如果是 0，直接 `break` 跳出整个 do-while——没许可可抢，调 poller 也是白跑（poller 内部 `tryAcquireIfReady` 阶段 2 也会查 `availablePermits` 立刻返回 `false`）。

为什么不直接让 poller 自己去判？因为遍历本身有开销：N 个 poller 就要触发 N 次 `tryAcquireIfReady`，每次都要查一遍

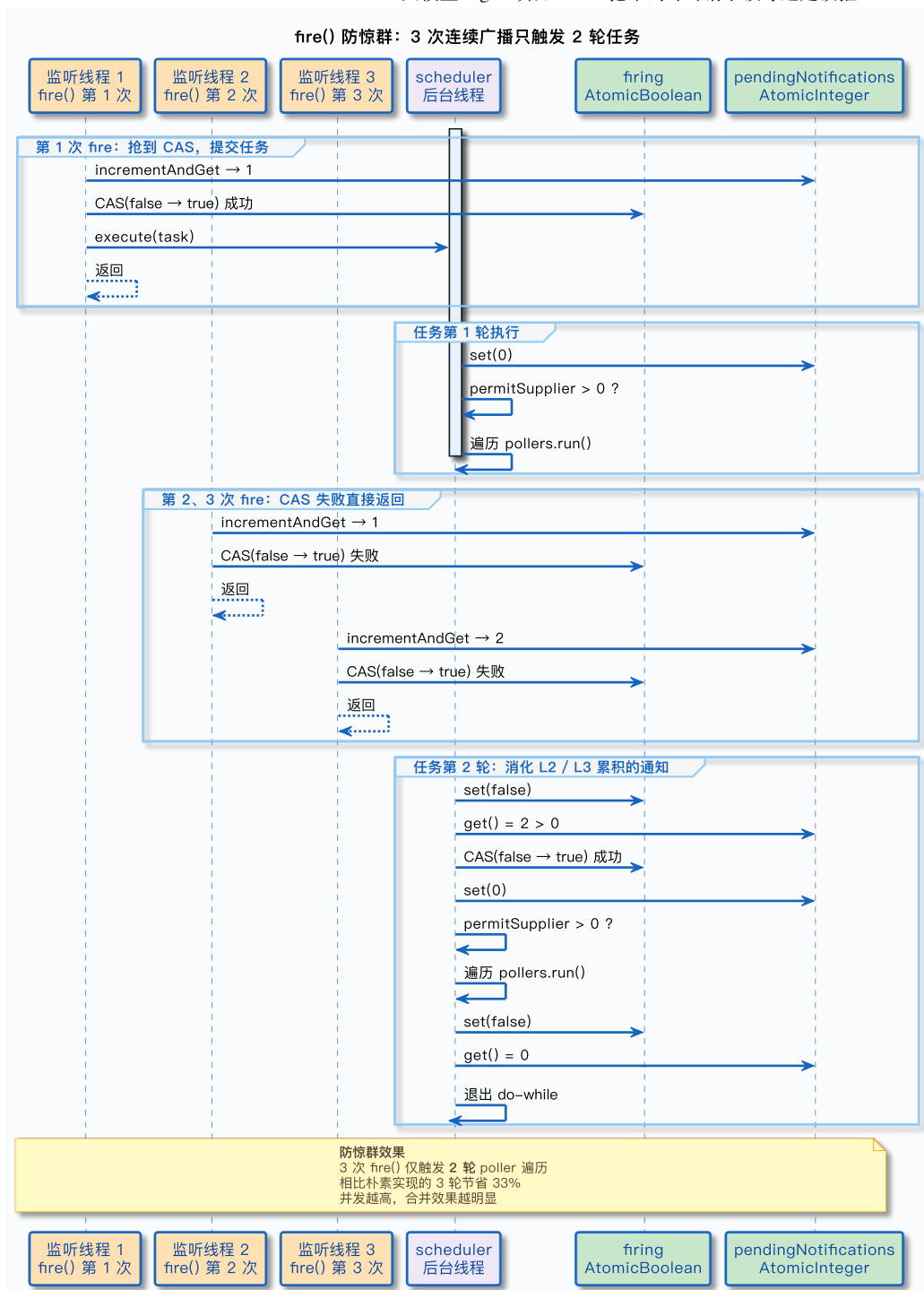
`availablePermits`——总共 N+1 次 Redis 查询。如果在 `fire()` 这一层先查一次，0 就跳过整轮——省下 N 次。

回到在线教育公司：节点 A 上有 7 个等待者。节点 B 释放许可时立刻有自己同节点的请求抢走了——广播到节点 A 时许可数已经回到 0 了。这次广播触发 `fire()`，`permitSupplier` 查到 0，直接跳过——节点 A 的 7 个 poller 一个都不跑。这种「广播来了但许可已被抢走」的场景在高并发下很常见，第三道防线显著降低了 Redis 调用密度。

注意这里用的是 `break` 而不是 `continue`。`break` 跳出 do-while 后 `finally` 块正常执行（`firing.set(false)` 释放执行权），但不再检查 `while` 条件——即使跑期间有新通知累积到 `pendingNotifications`，也不会再跑一轮。这是刻意的：许可为 0 时再跑一轮也是白跑，等下一次真正的 `release` 发新广播重新驱动就好。

3.4 一张图看 `fire()` 状态变化

把上面三层防御合起来用时序图展示一下「3 次连续 `fire()` 怎么被 1 个任务全部消化」：



实际跑起来比上面图简化的还要密集——一秒内可能来 10 次 `fire()`, 最终都被压成 1~2 轮。

4. `pendingNotifications.set(0)` 为什么在循环体开头

这一节单独拿出来讲, 因为是最容易写错的地方。

来看一段反例代码——把 `pendingNotifications.set(0)` 放循环体末尾:

```
// 反例: 先跑 poller 再清零 (漏通知)
executor.execute() -> {
    do {
        try {
            if (permitSupplier.getAsInt() <= 0) {
                pendingNotifications.set(0); // ❌ 漏在这里也不对
                break;
            }
            for (Runnable poller : pollers.values()) {
                poller.run();
            }
            pendingNotifications.set(0); // ❌ 错在这里
        } finally {

```

```
firing.set(false);
}
} while (pendingNotifications.get() > 0 && firing.compareAndSet(false, true));
});
```

这段反例错在哪？想象时序：

1. T=0ms：第一个 fire() 进来，pendingNotifications=1，CAS 成功，提交任务
2. T=1ms：任务开始遍历 pollers，跑 poller 1
3. T=10ms：跑 poller 5 期间，第二个广播来了 fire()，pendingNotifications=2，CAS 失败直接返回
4. T=15ms：所有 poller 遍历完
5. T=16ms：执行 pendingNotifications.set(0)——T=10ms 来的那条通知被一起清掉了
6. T=17ms：firing.set(false)
7. T=18ms：检查 pendingNotifications.get() = 0，不再循环

T=10ms 来的那条通知本应触发新一轮 poller 遍历，结果被「先跑后清」的写法吞掉了。如果这条通知对应的是「真的有许可释放、需要等待者去抢」，那就漏了。

正例（先清零再跑）：

```
// 正例：先清零再跑 poller
do {
    pendingNotifications.set(0); // ✅ 先清
    try {
        if (permitSupplier.getAsInt() <= 0) {
            break;
        }
        for (Runnable poller : pollers.values()) {
            poller.run();
        }
    } finally {
        firing.set(false);
    }
} while (pendingNotifications.get() > 0 && firing.compareAndSet(false, true));
```

同样的时序：

1. T=0ms：fire() 进来，pendingNotifications=1，CAS 成功提交任务
2. T=1ms：任务开始，pendingNotifications.set(0)——把入场的那一票 1 消化了
3. T=2ms：开始遍历 pollers
4. T=10ms：第二个广播来 fire()，pendingNotifications=1，CAS 失败返回
5. T=15ms：遍历完，firing.set(false)
6. T=16ms：检查 pendingNotifications.get() = 1 > 0，CAS 抢回执行权
7. T=17ms：开始第二轮，pendingNotifications.set(0) 消化 T=10ms 的那一票
8. T=18ms：遍历 pollers
9. T=25ms：完成，pendingNotifications=0，退出循环

T=10ms 来的通知正确地触发了第二轮遍历。语义可以这样理解：「set(0) 是『我接下来这一轮会消化掉之前的所有通知』的承诺。在这个承诺之前进来的通知归我；在这个承诺之后进来的通知归下一轮」。

类似的「先 reset 再 process」模式在 RxJava 的背压、Reactor 的 onBackpressureBuffer、Netty 的 EventLoop 都能看到。要点是把状态清零的时机放在「我承诺要处理」的那一刻，而不是「我处理完了」的那一刻——前者是「划清新旧通知的边界」，后者是「擦掉边界」。

等待超时：拒绝路径

1. Ticket.timeout() 驱动拒绝

回到 scheduleQueuePoll 的 deadline 分支：

```
if (System.currentTimeMillis() > ticket.deadline) {
    ticket.timeout();
    return;
}
```

进入这一分支说明等了超过 max-wait-seconds（默认 20s）还没轮到自己。ticket.timeout() 内部完成所有工作：

```
void timeout() {
    if (!state.compareAndSet(State.PENDING, State.TIMED_OUT)) {
        return;
    }
    cleanup();
    submitSafely(req.onTimeout(), "onTimeout");
}
```

三步走：

1. **CAS 抢占终态**：PENDING → TIMED_OUT。如果 CAS 失败（已被 cancel() 或 grant() 抢先），直接 return——不会重复触发拒绝回调
2. **cleanup()**：幂等清理——从 ZSET 移除自己、删除 entry 标记、释放许可（若持有，非 GRANTED 状态下）、广播通知、注销 poller、取消 future
3. **提交 onTimeout 回调**：在 Chat 场景里就是 () -> handleReject(question, conversationId, emitter)

handleReject 是 ChatQueueLimiter 里的方法：

```
private void handleReject(String question, String conversationId, SseEmitter emitter) {
    RejectedContext context = null;
    try {
        context = recordRejectedConversation(question, conversationId, resolveUserId());
    } catch (Exception ex) {
        log.warn("记录 reject 会话失败，仍向前端发送 DONE", ex);
    }
    sendRejectEvents(emitter, context);
}
```

两件事：写会话记忆 + 推 SSE 拒绝事件。注意 recordRejectedConversation 被 try-catch 包裹——记录会话失败不能阻塞 emitter 推送，否则前端永远收不到 DONE 事件，SSE 连接就挂死了。下面分别讲。

2. 把消息写进会话记忆

recordRejectedConversation 看代码：

```
private RejectedContext recordRejectedConversation(String question, String conversationId, String
    if (StrUtil.isBlank(question) || StrUtil.isBlank(userId)) {
        return null;
    }

    String actualConversationId;
    boolean isNewConversation;
    if (StrUtil.isBlank(conversationId)) {
        actualConversationId = IdUtil.getSnowflakeNextIdStr();
        isNewConversation = true;
    } else {
        actualConversationId = conversationId;
        isNewConversation = conversationGroupService.findConversation(actualConversationId, userId
    }

    memoryService.append(actualConversationId, userId, ChatMessage.user(question));
    String messageId = memoryService.append(actualConversationId, userId, ChatMessage.assistant(RE

    String title = Strings.EMPTY;
```

```
if (isNewConversation) {
    var conversation = conversationGroupService.findConversation(actualConversationId, userId)
    title = conversation != null ? conversation.getTitle() : Strings.EMPTY;
    if (StrUtil.isBlank(title)) {
        title = buildFallbackTitle(question);
    }
}
String taskId = IdUtil.getSnowflakeNextIdStr();
return new RejectedContext(actualConversationId, taskId, messageId, title);
}
```

2.1 为什么写记忆

这是产品决策不是技术决策。设想三种实现：

实现 A：超时直接关连接，不写任何东西。

用户视角：转圈圈突然消失，浏览器没收到任何 SSE 事件，前端只能显示「网络错误」。用户不知道发生了什么，会立刻刷新重试，新的请求又入队，雪上加霜。

实现 B：超时推一个 reject 事件，但不写记忆。

用户视角：能看到「系统繁忙」的提示，但刷新页面后这条对话不见了——没有用户问什么的记录，也没有系统回什么的记录。下次进入对话历史也找不到。这次失败像没发生过一样，用户的尝试和等待都没有任何痕迹。

实现 C（当前方案）：超时既推 reject 事件给前端实时显示，又把「用户的问题 + 系统繁忙的回复」写进会话记忆。

用户视角：当下能看到提示，刷新后会话历史里依然有这条记录——「我提了什么问题，系统给了什么回复」。下次进入对话能延续上下文，知道之前发生过什么。新会话还会建一个标题，在历史列表能找到这次对话。

C 方案的好处不止是用户体验，还有产品分析价值——后端可以统计哪些用户经常被限流、哪些时间段限流多、被拒的问题有什么特征，作为扩容决策的输入。

代码里有几个细节：

StrUtil.isBlank(question) || StrUtil.isBlank(userId) 直接 return null：没有问题或没有用户标识就没必要写记忆。userId 的解析由独立的 resolveUserId() 方法负责——先从 UserContext 取，取不到再从 Sa-Token 上下文里取，都取不到返回 null

conversationId 为空时跳过 findConversation 查询：刚生成的雪花 ID 不可能命中已有会话，直接标记 isNewConversation = true，省一次数据库查询

memoryService.append 两次：先写 user 消息（用户的问题），再写 assistant 消息（系统繁忙的回复），保持「一问一答」的结构

2.2 新会话还要建标题

```
String title = Strings.EMPTY;
if (isNewConversation) {
    var conversation = conversationGroupService.findConversation(actualConversationId, userId);
    title = conversation != null ? conversation.getTitle() : Strings.EMPTY;
    if (StrUtil.isBlank(title)) {
        title = buildFallbackTitle(question);
    }
}
}
```

如果是新会话，先回查 conversationGroupService.findConversation 拿到会话记录——因为前面 memoryService.append(USER) 内部会触发 conversationService.createOrUpdate，这个过程包含 LLM 生成标题。回查就能拿到生成结果。如果标题为空（LLM 生成失败或超时），调 buildFallbackTitle(question) 用问题前若干字截断作为兜底标题。

为什么需要兜底？这条对话的本质就是「请求失败了」——如果 LLM 生成标题也失败了，截断问题虽然简单粗暴，但保证用户在历史列表能识别出这是哪一条对话。

这里的兜底标题只针对「新会话超时拒绝」这种特殊情况。正常完成的对话会走 LLM 生成标题的路径，对话历史里的标题质量更高。

3. 推送拒绝事件序列

```
private void sendRejectEvents(SseEmitter emitter, RejectedContext rejectedContext) {
    SseEmitterSender sender = new SseEmitterSender(emitter);
    if (rejectedContext != null) {
        sender.sendEvent(SSEEventType.META.value(), new MetaPayload(rejectedContext.conversationId)
        sender.sendEvent(SSEEventType.REJECT.value(), new MessageDelta(RESPONSE_TYPE, REJECT_MESSAGE)
        sender.sendEvent(SSEEventType.FINISH.value(),
            new CompletionPayload(String.valueOf(rejectedContext.messageId), rejectedContext.t
```

```
    }
    sender.sendEvent(SSEEventType.DONE.value(), "[DONE]");
    sender.complete();
}
```

四个事件按顺序推：

- META：携带 conversationId 和 taskId，告诉前端「这是哪个会话的哪个任务」。前端拿到后可以更新 URL、绑定后续事件
- REJECT：携带 MessageDelta(type, content)，content 就是 REJECT_MESSAGE（系统繁忙，请稍后再试）。这个事件类型告诉前端「这次是失败的对话，可以做特殊渲染」
- FINISH：携带 messageId 和 title。messageId 是 assistant 那条消息的 ID，前端用这个 ID 关联到历史记录；title 是会话标题（新会话才有）
- DONE：内容固定 [DONE]，告诉前端「事件流结束了，可以关 EventSource 了」

最后调 sender.complete() 主动关 SSE 连接。

4. 正常完成 vs 队列超时拒绝

把正常完成和拒绝两种结局做个对比：

阶段	正常完成	队列超时拒绝
META	是	是
中间事件	MESSAGE × N（流式 token）	REJECT × 1（系统繁忙）
FINISH	带 messageId + title	带 messageId + title（拒绝消息已落库）
DONE	[DONE]	[DONE]
落库内容	完整模型回复	「系统繁忙，请稍后再试」
前端识别方式	event=message	event=reject

共用 META + FINISH + DONE 框架是有意为之——前端只需要多识别一个 reject 事件类型，剩下的逻辑（解析 META 拿 conversationId、解析 FINISH 拿 messageId、收到 DONE 关连接）和正常路径完全一致。这种「正常和异常共享主框架」的设计让前端代码不用大改。

拒绝事件用 MessageDelta(type, content) 而不是新定义一个类型，是因为这个结构能复用前端的渲染逻辑——把 content 渲染成一段文本就行，区别只是文本内容是「系统繁忙」而不是模型回复。

许可释放：从 Ticket.cleanup() 到 lease 兜底

许可释放是排队限流的另一个关键环节——许可如果发出去收不回来，整个限流系统会一直处于「无许可可发」的状态。这一节按从上到下的顺序讲三道防御。

1. cancelBinder + Ticket.cancel()：第一道防御

上一篇讲过，ChatQueueLimiter.enqueue() 通过 cancelBinder 把 ticket::cancel 绑定到 SseEmitter 的三个回调：

```
.cancelBinder(cancel -> {
    emitter.onCompletion(cancel);
    emitter.onTimeout(cancel);
    emitter.onError(e -> cancel.run());
})
```

这里的 cancel 就是 ticket::cancel。不管是业务正常完成（emitter.complete() 触发 onCompletion）、客户端关浏览器（TCP RST 触发 onCompletion 或 onError）、还是推送超时（onTimeout），都走同一条 cancel() → cleanup() 路径释放许可。Ticket.cancel() 做两件事：

```
void cancel() {
    state.compareAndSet(State.PENDING, State.CANCELLED);
```

```
cleanup();
}
```

`state.compareAndSet(State.PENDING, State.CANCELLED)`：如果还在排队（PENDING），CAS 抢占终态，阻止后续 `grant()` 或 `timeout()` 的回调执行

`cleanup()`：幂等清理资源

注意 `cancel()` 和 `timeout()/grant()` 有一个关键区别：CAS 失败后 `cancel()` 仍然调 `cleanup()`。为什么？因为 CAS 失败说明 Ticket 已经进入 GRANTED 或 TIMED_OUT 终态，但 `cleanup` 里的其他清理动作（移队、删 entry 标记、注销 poller、取消 future）仍然有意义。不过 `cleanup` 在 GRANTED 状态下不会释放许可——许可的生命周期已经交给 `grant()` 内部的 `try/finally` 包装，由业务执行结束后的 `releaseHeldPermit()` 负责归还。如果 `cleanup` 也去释放，就会在业务还在跑的时候把许可还给信号量，突破并发上限。

那 `grant()` 提交线程池失败怎么办？`grant()` 的 `catch` (`RejectedExecutionException`) 分支会显式调 `releaseHeldPermit()` 释放许可再调 `cleanup()` 做剩余清理，不会泄漏。

这是第一道防御——业务正常完成时许可可立刻归还。

2. Ticket.cleanup()：幂等清理的核心

`cleanup()` 是所有资源释放的汇聚点：

```
void cleanup() {
    boolean removed = false;
    try {
        removed = redissonClient.getScoredSortedSet(queueKey, StringCodec.INSTANCE).remove(requestId);
    } catch (Exception ex) {
        log.debug("[{}] 移除队列失败 (requestId={})", name, requestId, ex);
    }
    deleteEntryMarker(requestId);

    boolean releasedPermit = false;
    if (state.get() != State.GRANTED) {
        String permitId = permitRef.getAndSet(null);
        if (permitId != null) {
            releasePermitQuietly(permitId);
            releasedPermit = true;
        }
    }
    if (removed || releasedPermit) {
        publishQueueNotify();
    }
    unregisterFromNotifier();
    cancelFutureQuietly();
}
```

六个子操作，每个都幂等：

1. 从 ZSET 移除自己：`queue.remove(requestId)` 返回 `true/false` 表示是否真的移除了。不在队列里删除是空操作
2. 删除 entry 存活标记：`deleteEntryMarker(requestId)`。入队时写的 `rag:global:chat:entry:{requestId}` key，`cleanup` 时主动删掉——不留给 TTL 自然过期。这样后续 `Lua claim` 扫描队头窗口时不会看到一个已经不在队列但 entry 标记还活着的幽灵
3. 释放许可（仅在非 GRANTED 状态下）：`state.get() != State.GRANTED` 是关键守卫。如果 Ticket 已经进入 GRANTED 终态，许可的生命周期由 `grant()` 内部的 `try/finally` 包装接管——业务执行完毕（含异常）会在 `finally` 里调 `releaseHeldPermit()` 释放。如果 `cleanup` 也去释放，就会在业务还在跑的时候把许可还给信号量，等于让另一个请求拿到同一个 slot，突破并发上限。只有 PENDING / TIMED_OUT / CANCELLED 状态下 `cleanup` 才释放许可
4. 有变化才广播：只在「移了队 或 释放了许可」时才 `publishQueueNotify()`，避免无意义的空广播
5. 注销 poller：`pollNotifier.unregister(requestId)`
6. 取消定时器：`cancelFutureQuietly()`

和老版本相比，最大的改进是「所有释放逻辑收进一个方法」。老版本的释放分散在 `releaseOnce`（注册在 emitter 回调上）、`releasePermit`（在 `tryAcquireIfReady` 里）、超时分支（在 `scheduleQueuePoll` 里）等多个地方，每个地方都要手动拼凑 `queue.remove` + `permitRef.CAS` + `publishQueueNotify` + `unregister` + `cancelFuture` 的组合，容易遗漏。现在统一收进 `cleanup()`，所有路径只需要调一个方法。`releasePermitQuietly` 安静释放许可：

```
private void releasePermitQuietly(String permitId) {
    try {
        redissonClient.getPermitExpirableSemaphore(semaphoreKey).release(permitId);
    } catch (Exception ex) {
        log.debug("[{}] 释放 permit 失败 (可能已过期): {}", name, ex.getMessage());
    }
}
```

用 try-catch 包裹，忽略已过期等异常——这是防御性编程，因为 lease 过期后 Redis 端会自动回收许可，这时候再调 release 会失败，但这种失败是无害的。

3. 释放后必须广播

cleanup() 内部移队成功或释放许可后调 publishQueueNotify(), releaseHeldPermit() 释放许可后也调 publishQueueNotify()。这是必须的——一个许可空出来了，等待者要立刻被唤醒去抢。
少了这一步会怎样？最坏情况是所有等待者都靠 200ms 周期轮询发现许可空闲——实际响应延迟最高 200ms。配合 Pub/Sub 后正常情况几毫秒就能抢到，差距明显。
注意广播的语义是「队列状态/许可状态发生了变化，等待者来重新评估」，不是「这个许可分给你」。Pub/Sub 没有定向消费的概念，所有节点都收得到，所有等待者都被触发——但因为 ZSET FIFO + 信号量 acquire 的双重保护，最终只会有一个等待者拿到这个许可，其他人继续排队。这才是公平的。

4. lease 自动过期：终极兜底

前面两道防御都依赖业务代码主动释放——任何一个环节挂了（线程被强杀、JVM crash、机房断电），许可就再也回不来了。这种情况下 RPermitExpirableSemaphore 的 lease 是兜底：

```
private String tryAcquirePermit() {
    RPermitExpirableSemaphore sem = redissonClient.getPermitExpirableSemaphore(semaphoreKey);
    try {
        return sem.tryAcquire(0, leaseSecondsSupplier.getAsInt(), TimeUnit.SECONDS);
    } catch (InterruptedException ex) {
        Thread.currentThread().interrupt();
        return null;
    }
}
```

tryAcquire(0, lease, SECONDS) 拿到的许可绑定一个租约（默认 600s = 10 分钟）。Redis 端记录的不只是「N 个许可被占用」，而是「permitId=xxx 在 timestamp=yyy 之前必须归还」。如果到了 timestamp=yyy 还没归还，Redis 自动回收。
回到在线教育公司：节点 B 上的请求 1 拿到许可、跑业务跑到一半，机房 PDU 跳闸，整个节点宕机。许可永远不会通过 release 回来了。但 600 秒之后 Redis 自动回收这个许可——集群重新有了完整的 3 个许可，限流系统自我修复。
为什么 lease 设 600 秒？要比业务最长耗时大一些。RAG 业务最长可能跑 1~2 分钟（深度思考 + 长文档生成 + 重试）。600 秒 = 10 分钟，留了 5 倍的安全余量。

不能太短——业务还在跑，许可被自动回收了，新的请求拿到许可执行业务，最终业务完成调 release 时 Redis 端发现 permitId 不存在，安全失败，但这时候就是「同一时刻有 4 个业务在跑」（超过 max-concurrent=3）的情况。所以 lease 必须严格大于业务最长可能耗时。也不能太长——崩溃后许可几小时才回收，那段时间限流系统等于是降配了 1 个许可。

5. 各种异常路径的释放保证

把所有可能的释放路径整理成一张表：

异常场景	触发的事件	释放机制
业务正常完成	emitter.complete() → onCompletion → ticket.cancel()	grant() 包装的 try/finally → releaseHeldPermit() 释放许可 + 广播
客户端关闭浏览器	TCP RST → onCompletion 或 onError → ticket.cancel()	同上（业务仍在跑，cancel() 的 cleanup() 在 GRANTED 状态不释放 permit)
SSE 写超时	onTimeout → ticket.cancel()	同上
业务推送出错	onError → ticket.cancel()	同上

排队超时被拒	poller deadline → ticket.timeout()	cleanup() 释放许可（若有）+ 广播
拿到许可后被并发取消	grant() CAS 失败	permitRef.compareAndSet 释放 + 广播
提交 chatEntryExecutor 失败	RejectedExecutionException	grant() 内部 releaseHeldPermit() + cleanup() + 走 onTimeout 降级
业务异常但 SSE 连接还在	grant() 包装 Runnable 的 finally 块触发	releaseHeldPermit() 释放许可 + 广播
节点 JVM 崩溃	没有任何回调能触发	RPermitExpirableSemaphore lease 600s 后自动回收
网络断开节点失联	节点本地以为业务还在跑	同上，lease 兜底
调用方逻辑 bug 漏调 release	cleanup 路径漏掉	同上，lease 兜底

可以看到：GRANTED 状态下许可释放由 grant() 的 try/finally 包装接管，不经过 cleanup()；排队中的请求（PENDING / TIMED_OUT / CANCELLED）由 cleanup() 释放许可；「不可预期的灾难」由 lease 兜底。三层防御结合，许可绝不会永久泄漏。和老版本相比，最大的简化是所有可预期场景都收拢到 Ticket 状态机——cancel() / timeout() / grant() 三条路径各司其职，不再有 releaseOnce、releasePermit 等多个分散的释放逻辑。

优雅关闭：stop() + Spring destroyMethod

应用关闭那一瞬间，还在排队的请求要怎么收尾？daemon 线程虽然 JVM 退出会自动停，但留下「正在跑的 poller 半中断」「监听器还在接消息但应用要关」这种状态总是不优雅。
老版本在 ChatQueueLimiter 上用 @PreDestroy shutdown()。重构后关闭逻辑移到了 FairDistributedRateLimiter.stop()，通过 Spring 的 destroyMethod 触发：

```
@Bean(initMethod = "start", destroyMethod = "stop")
public FairDistributedRateLimiter chatRateLimiter(...)
```

stop() 方法：

```
public void stop() {
    if (!started.compareAndSet(true, false)) {
        return;
    }
    if (notifyListenerId != -1) {
        redissonClient.getTopic(notifyTopicKey).removeListener(notifyListenerId);
        notifyListenerId = -1;
    }
    scheduler.shutdown();
    awaitShutdown(scheduler);
    pollNotifier.clear();
}
```

1. stop() 的三步

最前面是 started.compareAndSet(true, false)——CAS 守卫，和 start() 的守卫对称。防止重复调 stop() 导致的重复 shutdown。

1.1 撤销 Pub/Sub 监听

```
if (notifyListenerId != -1) {
    redissonClient.getTopic(notifyTopicKey).removeListener(notifyListenerId);
    notifyListenerId = -1;
}
```

第一件事是从 Redis Topic 摘掉监听器。notifyListenerId 是 start() 时 addListener 的返回值，存下来就是为了关闭时反摘。摘除后把 notifyListenerId 置回 -1，标记已注销。
为什么先做这一步？因为应用要关了，再来新通知也没意义——后续的 scheduler.shutdown 已经不会再启动新的 poller 调度了。如果不先摘监听器，关闭过程中如果有广播来，会触发 pollNotifier.fire()，向正在关闭的 scheduler 提交任务，引发 RejectedExecutionException 之类的边缘异常。

1.2 停调度器并等待

```
scheduler.shutdown();
awaitShutdown(scheduler);
```

`scheduler.shutdown()`：拒绝接收新任务，但已经提交的任务继续跑完。

`awaitShutdown()` 等最多 3 秒：

```
private static void awaitShutdown(ScheduledExecutorService exec) {
    try {
        if (!exec.awaitTermination(3, TimeUnit.SECONDS)) {
            exec.shutdownNow();
        }
    } catch (InterruptedException ex) {
        exec.shutdownNow();
        Thread.currentThread().interrupt();
    }
}
```

3 秒够了——单个 poller 跑一次只要几毫秒到几十毫秒。如果 3 秒内还没跑完（极端情况，可能某个 poller 卡在 Redis 调用），`shutdownNow()` 强制中断。

`InterruptedException` 的处理是 Java 多线程的经典模式——捕获后立即重置中断标志位（`Thread.currentThread().interrupt()`），让上层调用者也能感知到中断信号。

1.3 清空 PollNotifier

```
pollNotifier.clear();
```

`PollNotifier.clear()` 内部就是一行 `pollers.clear()`——清空所有注册的 poller。因为 `PollNotifier` 不再有自己的线程池（复用 `scheduler`），不需要额外 shutdown。`scheduler.shutdown()` 在上一步已经执行完毕，所有调度任务都已停止。

关闭顺序有讲究：先撤监听器（不再接新事件），再停调度器（停轮询任务和广播触发任务），最后清空 `PollNotifier` 的 poller 注册表。这是从外到内的顺序——先切断外部输入源，再处理内部已经在跑的任务。反过来如果先停调度器再撤监听器，监听器可能继续触发 `fire()`，向已经关闭的 `scheduler` 提交任务，会抛 `RejectedExecutionException`。

2. Daemon 线程为什么还要主动关

这里有个常见疑问：所有线程池都设了 daemon，JVM 退出的时候不会自动杀掉所有 daemon 线程吗？为什么还要 `stop()` 主动关？

答案是「优雅 vs 粗暴」的差别。

JVM 退出时杀 daemon 线程是粗暴杀——线程在哪个指令上就停在哪。如果 poller 正好跑到 `tryAcquireIfReady` 的阶段 4，刚 `tryAcquire` 拿到 `permitId` 还没来得及 `permitRef.set(permitId)`，线程被杀，结果是 Redis 里有一个许可被占着但 Java 这边完全不知道是哪个 `permitId`。这个许可只能等 lease 600 秒后回收。

`stop()` 主动关是优雅关——`scheduler.shutdown()` 让正在跑的 poller 跑完整段逻辑（要么走完拿到许可的全流程，要么走完异常分支释放许可），不会留下半中断状态。`awaitTermination(3s)` 给了 3 秒时间让 poller 完成，3 秒够它们跑很多遍了。

退一万步说，即使 3 秒内还没跑完调 `shutdownNow` 强制中断，线程也是在「下一次中断检查点」才停——poller 里没有显式的中断检查，但 `tryAcquireIfReady` 内部有 Redis 调用（Redisson 的 `tryAcquire` 会响应 `InterruptedException`），所以最坏情况是当前 Redis 调用被中断、回到 `tryAcquireIfReady` 处理 `InterruptedException` 路径返回 null、poller 兜底分支跑完。

3. initMethod / destroyMethod vs @PostConstruct / @PreDestroy

老版本直接在 `ChatQueueLimiter` 上用 `@PostConstruct` / `@PreDestroy`。重构后改为在 `ChatRateLimiterConfig` 的 `@Bean` 注解里指定 `initMethod = "start"` 和 `destroyMethod = "stop"`。为什么要这么改？

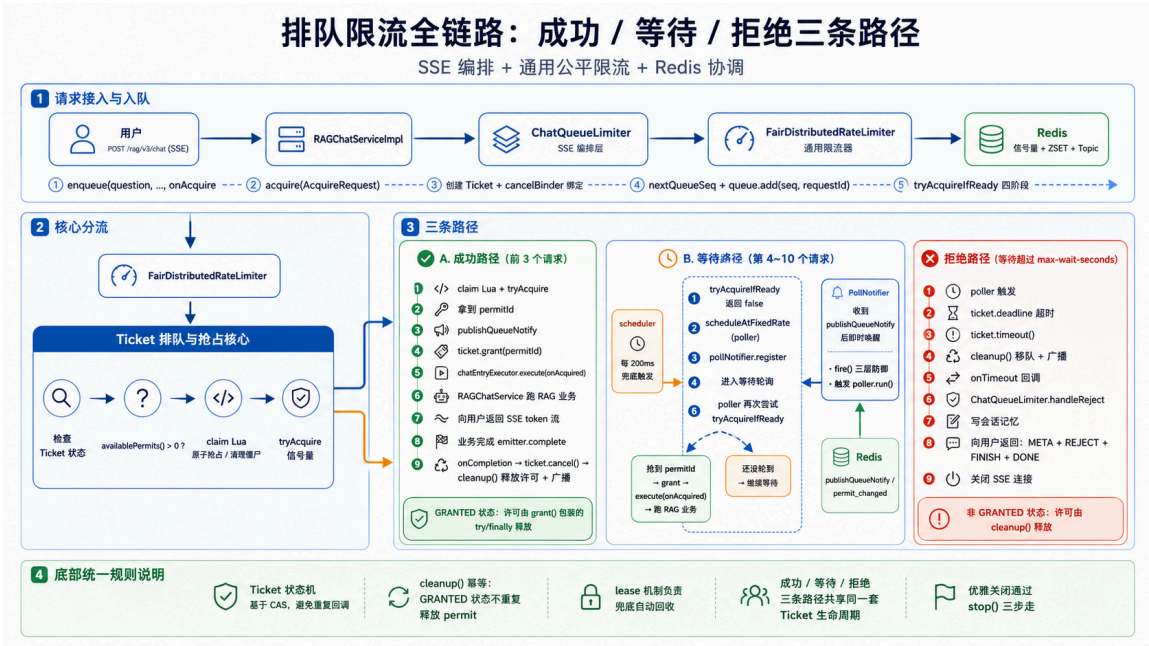
核心原因是 `FairDistributedRateLimiter` 是一个通用组件——它没有 `@Component` 注解，不依赖 Spring 的生命周期注解。通过 `initMethod` / `destroyMethod` 把生命周期管理交给上层的配置类（`ChatRateLimiterConfig`），组件本身保持 POJO。如果将来要在非 Spring 环境里用这个限流器（比如单元测试、其他框架），直接调 `start()` / `stop()` 就行。

Spring 应用的优雅停机是个独立话题。结合 `server.shutdown=graceful` + `spring.lifecycle.timeout-per-shutdown-phase=30s`，应用接到 `SIGTERM` 后会先停 Tomcat 接收新请求、等已有请求处理完、然后才触发 Bean 的 `destroyMethod`。所以等到 `FairDistributedRateLimiter.stop()` 跑的时候，已经没有新的 SSE 请求进来了，剩下的就是处理已经在排队的等待者。

全链路回看

讲完了同步路径（上篇）和异步路径（本篇）的所有细节，最后用一张大图把三条路径串起来。

1. 一张图看排队限流全链路



三条路径在这张图里清晰可见：

成功路径：入队 → 立即抢占 → ticket.grant() → 业务执行 → try/finally 的 releaseHeldPermit() 释放（最常见）

等待路径：入队 → 抢失败 → 调度（被广播或周期性触发） → 抢到 → ticket.grant() → 业务执行 → releaseHeldPermit() 释放

拒绝路径：入队 → 抢失败 → 调度 → deadline → ticket.timeout() → cleanup() 移队 → onTimeout 回调 → 写记忆 → 推 reject → 关连接

2. 配置参数调优经验

把上下两篇出现过的所有配置参数整理一张调优表：

参数	含义	默认值	调优思路
enabled	是否启用全局限流	true	生产开，本地 dev 可关
max-concurrent	集群总并发上限（信号量许可数）	3（演示）	按下游 LLM 并发上限设——SiliconFlow 5、自建 vLLM 看 GPU 显存
max-wait-seconds	单个请求最长等待时间	20	20~30 秒，超过用户体验已不可接受
lease-seconds	许可租约	600	严格大于业务最长耗时（含重试），生产建议 5~10 倍最长耗时
poll-interval-ms	等待者轮询间隔	200	50~500 之间，越短响应越快但 Redis 压力越大
awaitShutdown 超时	优雅关闭等待时间	3 秒	如果 poller 内有重型操作可调大；正常 3 秒够

调优时记住一个原则—— $\text{max-wait-seconds} \times 3 < \text{lease-seconds}$ 。前者是用户最长等待时间，后者是许可租约时间。后者必须显著大于前者，否则会出现「用户被拒之后业务还没跑完，许可被回收，然后业务跑完回调 release 失败」的奇怪现象（虽然不会出错，但日志会有告警）。

3. 一句话总结

排队限流的「复杂」是为了让业务代码足够「简单」。

回顾一下最终业务方代码长什么样：

```
chatQueueLimiter.enqueue(question, actualConversationId, emitter,
    () -> traceRunner.run(question, actualConversationId, taskId, callback,
        () -> chatPipeline.execute(ctx)));
```

业务方一行 enqueue() 把真正的业务包成 lambda 交给排队器，剩下的全部交给基础设施——5 个 Redis 数据结构（信号量、ZSET 队列、自增 seq、entry 存活标记、Pub/Sub Topic）+ 1 段 Lua 脚本（claim 原子操作）、1 个调度器线程池（scheduler，同时服务轮询和 PollNotifier 广播）、1 个业务线程池（chatEntryExecutor）、1 个 Ticket 状态机、1 个 PollNotifier（防惊群广播通知器）。这些东西都不会侵入业务代码。业务方不感知排队、不感知信号量、不感知 Pub/Sub、不感知 lease。两层架构把关注点分得很清楚——FairDistributedRateLimiter 管通用的分布式公平限流（信号量 + 队列 + Lua + 轮询 + 广播 + Ticket 状态机），ChatQueueLimiter 管 SSE 业务编排（emitter 生命周期绑定 + 超时写记忆 + 推 reject 事件）。这才是基础设施的价值——把复杂藏在背后，让业务代码保持清爽。

系列收尾：18 篇 AI 知识问答全回顾

到这里，AI 知识问答 18 篇正式收束。趁着记忆还热乎，把整个系列的脉络重走一遍。

1. 八阶段全景

第 1 篇用一张图给出了整个 RAG 链路的全景——StreamChatPipeline 把一次问答拆成八个阶段，按固定顺序执行，中间设三个短路点。这张图是后面 17 篇的地图，每一篇都在这张图上找到了自己的位置：



2. 每个子系列起到的作用

按子系列拆分一下，每个子系列在 RAG 工程中扮演的角色：

子系列	篇号	核心问题	起到的作用
全景导览	1	一次问答经历哪八个阶段	给出整个系列的地图
会话记忆	2~3	大模型没有记忆，多轮对话怎么不失忆	让连续追问保留上下文
查询重写	4	用户说的话和该搜的词不是一回事	把口语化问题变成检索系统能用的 query
意图识别树	5~9	四分类撑不住 20 个知识库，怎么精准路由	让模糊问题被分发到对的知识库
多通道检索	10~11	一次提问同时查多个库，最后只给模型几条	召回 + 精排，过滤噪声
MCP 工具调用	12~13	知识库答不了的问题交给工具去查	扩展模型的能力边界到外部系统
上下文组装	14	检索结果、工具数据、对话历史最终怎么拼	把所有材料拼成模型能用的 Prompt
流式生成	15	答案一个字一个字蹦出来怎么实现	让用户尽早看到首 token，体验更好
任务管控	16	用户点了停止，集群里发生了什么	让长任务能优雅停止

队列限流	17~18	10 个人同时问只有 3 个坑位，抢不到该等还是该拒	让系统在压力下不崩，有秩序地排队
------	-------	----------------------------	------------------

每个子系列单独看都解决一个具体的工程问题，合在一起构成了完整的 RAG 工程闭环。

3. 工程脉络的整体画像

站在工程角度看，整个 18 篇讲的是「一次知识问答的工程化全过程」。从用户敲下回车到最后一个 token 推送完毕，中间要经过：

入口防护层：@IdempotentSubmit 拦截重复提交、ChatQueueLimiter 排队限流（17~18 篇）。这一层保证系统在压力下不崩、用户的重复点击不会让自己排两次队。

会话上下文层：加载历史对话、做摘要压缩（2~3 篇）。把多轮对话变成模型能用的连贯上下文。

Query 处理层：改写、子问题拆分、意图识别（4~9 篇）。把用户的原始话语转换成检索系统和路由系统能用的结构化输入。

检索层：多通道并行检索 + 重排 + MCP 工具调用（10~13 篇）。从知识库捞文档、从外部系统拉数据。

生成层：上下文组装、流式生成、任务管控（14~16 篇）。把所有材料拼成 Prompt、调 LLM 流式生成、优雅处理用户停止。

把这五层叠起来，就是一个能上生产的 RAG 系统。任何一层缺失或者做得粗糙，都会在生产环境暴露出来：

- 没有限流？高频访问时刻被打爆
- 没有会话记忆？用户追问就失忆
- 没有 Query 改写？检索完全找不到东西
- 没有意图识别？所有问题都查所有知识库，慢且不准
- 没有重排？召回 30 条全塞给模型，Token 爆炸
- 没有 MCP？知识库答不了的问题就是没收录
- 没有流式？用户等几十秒看不到任何反馈
- 没有任务管控？用户点了停止系统继续烧 Token

这些没有就出事的工程细节，正是 RAG 系统从 demo 走向生产的关键。

至此，AI 知识问答 18 篇正式收束。从第 1 篇的八阶段全景，到中间的记忆 / 改写 / 意图 / 检索 / MCP / 组装 / 流式 / 取消，到最后两篇的限流，整个 RAG 知识问答的工程脉络已经完整地铺开。希望你读完之后，再看 RAG 系统的任何一篇技术文章或者任何一段源码，脑子里都能立刻定位到「这是哪一层、解决什么问题、和其他层怎么衔接」。

工程世界没有银弹，但有规律。RAG 系统的复杂度不在某一个技术点，而在如何把这些技术点拼成一个稳定运行的整体。

